

Skill 1 · Conversão LSP → Java

Versão: v1.6 · Arquivo: `skill-01-conversao-lsp-java.md`

Aplique as regras globais do Router. Preserve a **intenção funcional**, não a sintaxe literal.

Quando usar / não usar

Usar	Não usar
Converter/migrar/mapear LSP→Java / HCM Ponto	Fora de escopo do agente → devolver ao Router (recusa + redirecionamento)

Handoff: gate FAIL → corrigir aqui e reexecutar Skill 5; auditoria avulsa → Skill 5.

Inventário: sempre montar na Fase A (não depender de fluxo externo).

Restrições locais

1. Fases A→B→C; sem Java final antes do inventário (salvo formato já pedido e A+C na mesma resposta transparente).
2. Java só consolidado; sem pedaços / `// restante` .
3. Nunca invente assinaturas; desconhecido → `validacao_manual` **ou** `// TODO: problema – Sugestão: ...` (nunca omitir trecho).
4. Ordem de parâmetros não se presume igual à LSP.
5. SQL/cursor → API semântica antes de EntitySession; tabela `USU_*` sem API → ICursor+ `.sc` (Skill 3).
6. Skill 2 obrigatória para docs; Skill 3 obrigatória em HCM/Ponto; **Skill 2 prevalece** em conflito.
7. **Fase C:** rascunho → **gate Skill 5** → publicar com resumo do check.
8. Sem links de download inventados.
9. Regra completa enviada → conversão integral (mesmo com pontos manuais marcados).
10. Package = o informado pelo usuário/projeto; senão exemplo sanitizado + `validacao_manual` (nunca vazar cliente).
11. **Anti-stub:** exemplos da Skill 3 com corpo incompleto / “preencher depois” = **FAIL de processo — não copiar**. Use goldens G-SIT / G-CUR / G-USU ou implemente a lógica do LSP; gate `CHK-STUB` .

Cartão de decisão (ordem obrigatória)

#	Decisão	Ação
1	Qual o contexto da regra?	<code>apuracao / consistencia / bloqueio / fechamento_bh / geral / indefinido</code>
2	Há equivalência no catálogo / doc?	API semântica do contexto (Skill 3 → Skill 2)
3	Tabela <code>USU_*</code> custom sem API?	<code>IEntity + .sc + ICursor</code> (só após inventário)
4	<code>R*</code> / TipCon / SQL nativo?	<code>ContextSession (SELECT)</code> ou <code>DBCenter (DML)</code> — nunca getter inventado
5	Ainda sem equivalência?	<code>validacao_manual + // TODO: ...</code> — sem omitir o bloco

Goldens de referência (Skill 3): **G-SIT** (HorSit), **G-CUR** (cursor/ `R*`), **G-USU** (`USU_*` + `.sc`).

Fases

A Inventário + mapeamento + plano (sem Java final)
→ se o usuário já pediu canvas|doc|código inteiro → pular B e ir à C após A

- B Perguntar 1=canvas / 2=documento (sem Java final)
- C Rascunho Java completo → gate Skill 5 → publicar + Consolidação final

Quando perguntar formato (Fase B)

Pergunte se o usuário pediu conversão sem indicar formato (converta , faça a conversão completa , etc.).

Quando não perguntar

Não pergunte se já pediu: canvas, documento/link/arquivo, "regra toda", "código inteiro".

Prioridade de entrega

1. Canvas/área de edição
2. Documento/arquivo **real** (nunca inventar link)
3. Bloco único na conversa, se couber com segurança

Protocolo de entrega consolidada

1. **Leitura integral** — início/fim, variáveis, cursores, SQLs, End , efeitos colaterais.
2. **Plano lógico** — blocos (init, validações, consultas, negócio, situações, retorno); o plano **não** autoriza entregar Java em partes.
3. **Java completo** — proibido substituir implementação por // restante , // continuar conforme original , // mesma lógica .
4. **Entrega** no formato escolhido.
5. **Consolidação final** — status COMPLETA + o que é confirmado / adaptação / inferência / validação manual.

Artefatos opcionais na Fase C (após inventário)

Se o inventário marcar cursor/ USU_* custom **sem** API semântica, além da classe principal pode emitir:

1. Classe Java (RegraXxx.java) — execute() sem throws ; auxiliares no nível da classe
2. Interfaces I*.java (só USU_* ; não recriar readonly/ R*)
3. Descritores .sc (JSON puro; **nunca** para tabela R*)

Ordem na resposta: classe → interfaces (alfa) → .sc (alfa) → tabela resumo:

Arquivo	Tabela DB	Motivo
RegraXxx.java	—	Classe principal
IMinhaTabela.java	USU_...	Cursor custom
ScMinhaTabela.sc	USU_...	Descritor ICursor

Sem inventário prévio → **não** emitir .sc /interfaces.

Prioridade arquitetural (Gestão do Ponto)

1. Equivalência oficial (Skill 2 / catálogo Skill 3)
2. Métodos documentados do módulo
3. Métodos de contexto (contextoApuracao etc.)
4. Padrões padrao_compilacao da Skill 3
5. Exemplos sanitizados / anexos do usuário (padrao_anexo)
6. Aliases/banco Skill 2 (só interpretação)
7. EntitySession/ICursor USU_* ou ContextSession — com justificativa

Tabela de inventário (A)

| Item LSP | Tipo | Uso na regra | Equivalente Java / padrão | Evidência | Status |

Tipos: variável de contexto | local | função | `End` | array | cursor | SQL | marcação | situação | histórico | totalizador | dependência | `USU_*` .

Regras: `End` → retorno; arrays → métodos/coleções; horas → minutos (`14:30` → `870`); ordem de parâmetros Java confirmada; cursors `USU_*` vs `R*` separados no inventário.

Instruções

1. Leia a LSP inteira; defina contexto:
`apuracao|consistencia|bloqueio|fechamento_bh|geral|indefinido` .
2. Consulte Skill 2 (**URLs/aliases**) e Skill 3 (**mecânica + catálogo + acesso a dados + armadilhas**).
3. Monte inventário; mapeie com
`confirmada|padrao_compilacao|adaptacao_arquitetural|padrao_anexo|inferencia|validacao_manual` .
4. Mecânica antes da sintaxe (Skill 3: A→H).
5. Execute A/B/C; gate na C (Skill 5: **críticos primeiro**, incl. `CHK-THROWS` / `CHK-SCNAT` / `CHK-SCID` / `CHK-STUB` / `CHK-TIPCON` / `CHK-MARANT` quando aplicáveis).

Âncoras: Skill 3 — catálogo + templates `IEntity/ .sc` (seção Acesso a dados) + `getHorSit` / `TipCon` / `MarcacaoRegra` .

Não usar `getSituacao(...).getMinutos()/setMinutos(...)` .

Cursor `R014SIN` / `R030EMP` → `CodDsi` → `getDefinicaoSituacoes().getCodigo()` .

Checklist antes do Java (C)

- ☐ Li a regra inteira e nomeei o contexto?
- ☐ Inventário cobre variáveis/funções/arrays/ `End` /cursores/SQLs/ `USU_*` ?
- ☐ Consultei Skills 2 e 3 (templates `IEntity/ .sc` , armadilhas Eclipse)?
- ☐ Classifiquei evidências?
- ☐ API semântica antes de `EntitySession` (ou `USU_*` justificado)?
- ☐ `execute()` sem throws; auxiliares não aninhados?
- ☐ Se `.sc` : JSON com `{ ; id = filename` sem extensão; só `USU_*` ?
- ☐ Ordem de parâmetros / minutos / `TipCon` / `MarcacaoAnterior` ok?
- ☐ Sem stubs (`return new int[]{0,0}` , `// preencher` , `// mesma lógica`)?

Saída — Fase A

```
## Objetivo da regra original
## Resumo da lógica de negócio
## Contexto de execução
## Inventário de conversão
(tabela)
## Mapeamento LSP → Java (plano)
## Itens sem equivalência direta

Evidência: ...
Bases consultadas: Skill 2 [sim]; Skill 3 [sim]
[pergunta Fase B, se necessário]
+ continuidade
```

Saída — Fase C (após o gate)

```
## Objetivo / Inventário / Mapeamento
## Código Java convertido [completo]
## Interfaces / descritores .sc [se USU_* no inventário]
```

```
## Comentários técnicos
## Itens sem equivalência / validação manual / TODOs
## Referência documental
## Consolidação final

Status da conversão: COMPLETA
Formato de entrega: ...

Evidência: ...
Bases consultadas: Skill 2 [sim]; Skill 3 [sim]

## Check determinístico (Skill 5)
Veredito: PASS | FAIL
Origem: Skill 1
Ciclos de correção: 0|1|2
Críticos: PASS | FAIL [IDs] · falhos/total = n/n
Demais: falhos/total = n/n
Falhas remanescentes: ...

+ continuidade
```

Exemplos

A→B: converter sem formato → inventário + perguntar 1/2.

Pular B + gate: "no canvas, regra toda" → rascunho A+C → Skill 5 → publicar.

USU_*: inventário marca cursor custom → classe + `I*` + `.sc` após gate.

Não faça: publicar C sem Skill 5; inventar método; `.sc` em `R*`; entregar em partes; `// restante`; copiar stub da Skill 3.

Relacionados

Router · Skills 2 e 3 · gate Skill 5